Project_code

Main source files from ATRIQUET (curated subset, not full codebase).

backend/main.py

```
from fastapi import FastAPI, File, UploadFile, Form, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from contextlib import asynccontextmanager
import os
from dotenv import load_dotenv
import logging
import base64
import binascii

from services.openrouter_service import OpenRouterService
from services.recommendation_engine import RecommendationEngine
from services.groq_service import GroqService
from services.clothing_image_service import ClothingImageService
from services.gemini_vton_service import GeminiVTONService
from utils.cache_manager import CacheManager
from models.schemas import UserAttributes
from pydantic import BaseModel


import sys
from pathlib import Path

# Add parent directory to path
sys.path.append(str(Path(__file__).parent.parent))

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)


def _unwrap_nested_base64(payload: str) -> str:
    current = payload.strip().strip('"').strip("'")

    if current.startswith("data:image/"):
        current = current.split(",", 1)[1] if "," in current else current

    for _ in range(2):
        try:
            raw = base64.b64decode(current, validate=False)
        except (binascii.Error, ValueError):
            break

        # If the decoded bytes are already an image, stop unwrapping.
        if raw.startswith(b"\x89PNG\r\n\x1a\n") or raw.startswith(b"\xff\xd8\xff") or raw.startswith((b"GIF87a",
b"GIF89a")) or (raw.startswith(b"RIFF") and len(raw) > 12 and raw[8:12] == b"WEBP"):
            break

        try:
            decoded_text = raw.decode("ascii").strip().strip('"').strip("'")
        except UnicodeDecodeError:
            break

        base64_chars = set("ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/=\r\n")
        if decoded_text and all(ch in base64_chars for ch in decoded_text):
            current = decoded_text
            continue

        break

    return current


def _guess_image_mime_from_base64(image_base64: str) -> str:
    payload = _unwrap_nested_base64(image_base64)
    if payload.startswith("data:image/"):
        try:
            return payload.split(";", 1)[0].split(":", 1)[1]
        except (IndexError, ValueError):
            return "image/png"

    if "," in payload and payload.split(",", 1)[0].endswith(";base64"):
        payload = payload.split(",", 1)[1]
```

```
    try:
        raw = base64.b64decode(payload, validate=False)
    except (binascii.Error, ValueError):
        return "image/png"

    if raw.startswith(b"\x89PNG\r\n\x1a\n"):
        return "image/png"
    if raw.startswith(b"\xff\xd8\xff"):
        return "image/jpeg"
    if raw.startswith((b"GIF87a", b"GIF89a")):
        return "image/gif"
    if raw.startswith(b"RIFF") and len(raw) > 12 and raw[8:12] == b"WEBP":
        return "image/webp"

    return "image/png"


def _to_data_url(image_base64: str) -> str:
    payload = _unwrap_nested_base64(image_base64)
    if payload.startswith("data:image/"):
        return payload

    mime = _guess_image_mime_from_base64(payload)
    if "," in payload and payload.split(",", 1)[0].endswith(";base64"):
        payload = payload.split(",", 1)[1]
    return f"data:{mime}base64,{payload}"

# Load environment variables from one authoritative location.
# Prefer repo-root .env to avoid stale keys from parent shell or other files.
project_root = Path(__file__).parent.parent
env_candidates = [
    project_root / ".env",
    project_root / "config" / ".env",
    project_root.parent / ".env",
    Path.cwd() / ".env",
    Path(__file__).parent / ".env",
]

loaded_env = None
for env_path in env_candidates:
    if env_path.exists():
        load_dotenv(env_path, override=True)
        loaded_env = env_path
        logger.info(f"Loaded environment from: {env_path}")
        break

if loaded_env is None:
    logger.warning("No .env file found in expected locations.")

@asynccontextmanager
async def lifespan(app: FastAPI):
    logger.info("Starting ATRIQUET Fashion Recommendation System")
    yield
    logger.info("Shutting down ATRIQUET")

app = FastAPI(
    title="ATRIQUET Fashion Recommendation API",
    description="Simple outfit analyzer",
    version="2.0.0",
    lifespan=lifespan
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Initialize Services
openrouter_api_key = os.getenv("OPENROUTER_API_KEY")
openrouter_service = OpenRouterService(api_key=openrouter_api_key) if openrouter_api_key else None
if not openrouter_service:
    logger.warning("OPENROUTER_API_KEY not found. OpenRouter quick analysis endpoint will use Groq instead.")
```

```
cache_manager = CacheManager()
groq_service = GroqService(api_key=os.getenv("GROQ_API_KEY"))
clothing_image_service = ClothingImageService()

# Initialize Gemini VTON Service (optional - only if key is provided)
vton_service = None
if os.getenv("GEMINI_API_KEY"):
    gemini_model = os.getenv("GEMINI_MODEL", "nano-banana-pro-preview")
    vton_service = GeminiVTONService(
        api_key=os.getenv("GEMINI_API_KEY"),
        model=gemini_model
    )
    logger.info(f"Gemini VTON model: {gemini_model}")
    logger.info("Gemini VTON Service initialized")
else:
    logger.warning("GEMINI_API_KEY not found. VTON generation will be unavailable.")

recommendation_engine = RecommendationEngine(
    openrouter_service=openrouter_service,
    groq_service=groq_service,
    cache_manager=cache_manager,
    clothing_image_service=clothing_image_service,
    vton_service=vton_service
)

# Request model for clothing images
class ClothingImageRequest(BaseModel):
    top: str
    bottom: str
    shoes: str

@app.get("/")
async def root():
    return {
        "message": "ATRIQUET Fashion Recommendation API",
        "version": "2.0.0",
        "status": "operational"
    }

@app.get("/health")
async def health_check():
    return {
        "status": "healthy"
    }

@app.post("/api/clothing-images")
async def get_clothing_images(request: ClothingImageRequest):
    """
    Fetch clothing images from the web for recommended outfit items.
    """
    try:
        logger.info(f"Fetching clothing images for: top={request.top[:30]}..., bottom={request.bottom[:30]}...")

        images = await clothing_image_service.get_outfit_images(
            top=request.top,
            bottom=request.bottom,
            shoes=request.shoes
        )

        return images

    except Exception as e:
        logger.error(f"Error fetching clothing images: {str(e)}")
        raise HTTPException(status_code=500, detail=str(e))

@app.post("/api/analyze")
async def analyze_outfit(
    image: UploadFile = File(...),
    occasion: str = Form("casual"),
    style: str = Form("modern"),
    include_brands: bool = Form(False)
):
    """
    Full outfit analysis and recommendation with Avatar visualization.
```

```
    """
    try:
        logger.info(f"Processing request - Occasion: {occasion}, Style: {style}")

        # Read and encode image
        image_data = await image.read()

        # Basic validation
        if len(image_data) > 10 * 1024 * 1024:  # 10MB limit
            raise HTTPException(status_code=400, detail="Image too large. Max 10MB.")

        image_base64 = base64.b64encode(image_data).decode('utf-8')

        # Generate recommendations
        response = await recommendation_engine.generate_recommendations(
            image=image_base64,
            occasion=occasion,
            style=style,
            include_brands=include_brands
        )

        return response


    except HTTPException as he:
        raise he
    except Exception as e:
        import traceback
        logger.error(f"Error processing request: {str(e)}")
        logger.error(f"Traceback: {traceback.format_exc()}")
        raise HTTPException(status_code=500, detail=str(e))

@app.post("/api/quick-analyze")
async def quick_analyze(
    image: UploadFile = File(...)
):
    """
    Quick analysis endpoint - just tells you what the person is wearing.
    """
    try:
        image_data = await image.read()

        # Basic validation
        if len(image_data) > 10 * 1024 * 1024:
            raise HTTPException(status_code=400, detail="Image too large. Max 10MB.")

        image_base64 = base64.b64encode(image_data).decode('utf-8')

        prompt = "Briefly describe what this person is wearing. Just the basics: items and colors."

        if openrouter_service:
            description = await openrouter_service.describe_outfit(image_base64, prompt)
        else:
            description = await groq_service.analyze_image(image_base64, prompt)

        return {
            "success": True,
            "description": description
        }

    except Exception as e:
        logger.error(f"Error in quick analysis: {str(e)}")
        raise HTTPException(status_code=500, detail=str(e))

@app.get("/api/occasions")
async def get_occasions():
    """Return available occasion types."""
    return {
        "occasions": [
            "casual",
            "formal",
            "business",
            "party",
            "wedding",
            "date_night",
```

```
            "beach",
            "sports",
            "travel"
        ]
    }

@app.get("/api/styles")
async def get_styles():
    """Return available style types."""
    return {
        "styles": [
            "minimalist",
            "streetwear",
            "classic",
            "bohemian",
            "preppy",
            "edgy",
            "romantic",
            "athletic",
            "vintage",
            "modern"
        ]
    }

@app.post("/api/generate-avatar")
async def generate_avatar(
    outfit_description: str = Form(...),
    body_type: str = Form("average"),
    skin_tone: str = Form("medium"),
    style: str = Form("modern")
):
    """
    Generate a fashion avatar wearing the described outfit using Cloudflare Workers AI.

    Args:
        outfit_description: Description of the outfit (e.g., "blue jeans, white t-shirt, sneakers")
        body_type: Body type descriptor (slim, average, athletic, curvy)
        skin_tone: Skin tone descriptor (fair, medium, tan, dark)
        style: Fashion style (modern, vintage, streetwear, etc.)

    Returns:
        Base64 encoded image of the generated avatar
    """
    try:
        if not vton_service:
            raise HTTPException(
                status_code=503,
                detail="Avatar generation service is not configured. Please add GEMINI_API_KEY to your .env file."
            )

        logger.info(f"Generating avatar: {outfit_description[:50]}...")

        # Generate the avatar image
        image_base64 = await vton_service.generate_outfit_visualization(
            outfit_description=outfit_description,
            body_type=body_type,
            skin_tone=skin_tone,
            style=style
        )

        return {
            "success": True,
            "image": _to_data_url(image_base64),
            "message": "Avatar generated successfully"
        }

    except HTTPException as he:
        raise he
    except Exception as e:
        logger.error(f"Error generating avatar: {str(e)}")
        raise HTTPException(status_code=500, detail=f"Failed to generate avatar: {str(e)}")

@app.post("/api/generate-custom-image")
async def generate_custom_image(
    prompt: str = Form(...),
```

```
    width: int = Form(512),
    height: int = Form(768),
    model: str = Form("gemini-3-pro-image-preview")
):
    """
    Generate a custom image using Gemini image generation.

    Args:
        prompt: Full text prompt for image generation
        width: Image width (256-2048)
        height: Image height (256-2048)
        model: Model to use (currently fixed to gemini-3-pro-image-preview)

    Returns:
        Base64 encoded generated image
    """
    try:
        if not vton_service:
            raise HTTPException(
                status_code=503,
                detail="Image generation service is not configured. Please add GEMINI_API_KEY to your .env file."
            )

        # Validate dimensions
        if not (256 <= width <= 2048) or not (256 <= height <= 2048):
            raise HTTPException(
                status_code=400,
                detail="Width and height must be between 256 and 2048 pixels"
            )

        logger.info(f"Generating custom image: {prompt[:50]}...")

        image_base64 = await vton_service.generate_avatar(
            prompt=prompt,
            width=width,
            height=height,
            model=model
        )

        return {
            "success": True,
            "image": _to_data_url(image_base64),
            "message": "Image generated successfully"
        }

    except HTTPException as he:
        raise he
    except Exception as e:
        logger.error(f"Error generating custom image: {str(e)}")
        raise HTTPException(status_code=500, detail=f"Failed to generate image: {str(e)}")

@app.post("/api/generate-outfit-on-person")
async def generate_outfit_on_person(
    image: UploadFile = File(...),
    garment_image: UploadFile = File(None),
    outfit_description: str = Form(""),
    body_type: str = Form("average"),
    skin_tone: str = Form("medium"),
    gender: str = Form("person")
):
    """
    Generate image of the SAME person wearing a NEW outfit using Gemini VTON.

    Args:
        image: Original photo of the person
        garment_image: Optional garment/product image to transfer onto the person
        outfit_description: Optional text instruction for garment styling
        body_type: Body type descriptor (slim, average, athletic, curvy)
        skin_tone: Skin tone descriptor (fair, medium, tan, dark)
        gender: Gender descriptor (man, woman, person)

    Returns:
        Base64 encoded image showing the same person in the new outfit
    """
    try:
```

```
        if not vton_service:
            raise HTTPException(
                status_code=503,
                detail="Outfit generation service is not configured. Please add GEMINI_API_KEY to your .env file."
            )

        logger.info("Generating new outfit on person using Gemini VTON...")

        # Read and encode the original image
        image_data = await image.read()

        if len(image_data) > 10 * 1024 * 1024:
            raise HTTPException(status_code=400, detail="Image too large. Max 10MB.")

        image_base64 = base64.b64encode(image_data).decode('utf-8')

        body_attrs = {
            'body type': body_type,
            'skin tone': skin_tone,
            'gender': gender
        }

        result_image_base64 = None

        if garment_image is not None:
            garment_data = await garment_image.read()
            if len(garment_data) > 10 * 1024 * 1024:
                raise HTTPException(status_code=400, detail="Garment image too large. Max 10MB.")

            garment_base64 = base64.b64encode(garment_data).decode('utf-8')

            person_mime = image.content_type or "image/jpeg"
            garment_mime = garment_image.content_type or "image/jpeg"

            result_image_base64 = await vton_service.generate_outfit_from_garment(
                original_image_base64=image_base64,
                garment_image_base64=garment_base64,
                body_attributes=body_attrs,
                person_mime_type=person_mime,
                garment_mime_type=garment_mime,
                extra_instruction=outfit_description,
            )
        else:
            if not outfit_description.strip():
                raise HTTPException(status_code=400, detail="Provide garment_image or outfit_description.")

            result_image_base64 = await vton_service.generate_outfit_on_person(
                original_image_base64=image_base64,
                outfit_description=outfit_description,
                body_attributes=body_attrs
            )

        return {
            "success": True,
            "image": _to_data_url(result_image_base64),
            "message": "Outfit visualization generated successfully"
        }

    except HTTPException as he:
        raise he
    except Exception as e:
        raw_error = str(e)
        sanitized_error = raw_error.replace("inline_data=Blob(data='", "inline_data=Blob(data='[truncated]")
        if len(sanitized_error) > 400:
            sanitized_error = f"{sanitized_error[:400]}..."

        logger.exception("Error generating outfit on person")
        raise HTTPException(status_code=500, detail=f"Failed to generate outfit: {sanitized_error}")

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(
        "main:app",
        host=os.getenv("HOST", "0.0.0.0"),
        port=int(os.getenv("PORT", 8000)),
```

```
        reload=os.getenv("DEBUG", "True") == "True"
    )
```

backend/models/schemas.py

```
from pydantic import BaseModel, Field
from typing import List, Optional, Dict, Any
from enum import Enum

class BodyType(str, Enum):
    RECTANGLE = "rectangle"
    TRIANGLE = "triangle"
    INVERTED_TRIANGLE = "inverted_triangle"
    HOURGLASS = "hourglass"
    OVAL = "oval"

class SkinTone(str, Enum):
    FAIR = "fair"
    LIGHT = "light"
    MEDIUM = "medium"
    OLIVE = "olive"
    TAN = "tan"
    DEEP = "deep"

class UserAttributes(BaseModel):
    height: Optional[str] = None
    skin_tone: Optional[str] = None
    hair_style: Optional[str] = None
    hair_color: Optional[str] = None
    gender: Optional[str] = "Male" # Default or required? Let's make it optional but default to Male for now as
example
    body_shape: Optional[str] = None

class RecommendationRequest(BaseModel):
    occasion: str
    style: str
    user_attributes: Optional[UserAttributes] = None
    include_brands: bool = False

class PhysicalAnalysis(BaseModel):
    body_type: str
    proportions: Dict[str, str]
    skin_tone: str
    skin_undertone: str
    recommended_colors: List[str]
    colors_to_avoid: List[str]
    flattering_silhouettes: List[str]
    style_notes: str
    # User attributes for avatar generation
    user_attributes: Optional[Dict[str, str]] = None

class OutfitRecommendation(BaseModel):
    outfit_name: str
    description: str
    top: str
    bottom: str
    shoes: Optional[str] = None
    accessories: List[str]
    colors: List[str]
    brands: Optional[List[str]] = None
    rationale: str
    styling_tips: List[str]
    confidence_score: float
    # Avatar image URL (deprecated - use clothing images instead)
    avatar_image_url: Optional[str] = None
    why_it_works: Optional[str] = None  # For frontend display
    match_score: Optional[int] = None  # Percentage match
    # Real clothing images from web
    top_image_url: Optional[str] = None
    bottom_image_url: Optional[str] = None
    shoes_image_url: Optional[str] = None

class RecommendationResponse(BaseModel):
    success: bool
    physical_analysis: Optional[PhysicalAnalysis] = None # Made optional as we might rely less on explicit analysis
    recommendations: List[OutfitRecommendation]
    general_styling_tips: List[str]
```

```
    occasion: str
    style: str
    processing_time: float
    used_api: str

    # New Validation Fields
    is_appropriate: bool
    critique: Optional[str] = None
    improvement_suggestions: Optional[str] = None
    generated_avatars: List[Dict[str, Any]] = [] # Structured avatar payloads for multiple outfits
    avatar_url: Optional[str] = None # Keep for backward compatibility or primary avatar

class QuickAnalysisResponse(BaseModel):
    success: bool
    body_type: str
    skin_tone: str
    recommended_colors: List[str]
    flattering_styles: List[str]
    processing_time: float
```

backend/services/recommendation_engine.py

```
import logging
import time
from typing import Dict, Any
import json
from models.schemas import RecommendationResponse, PhysicalAnalysis, OutfitRecommendation, UserAttributes
from typing import Optional
from services.clothing_image_service import ClothingImageService
from services.openrouter_service import OpenRouterService
from services.groq_service import GroqService
from utils.cache_manager import CacheManager

logger = logging.getLogger(__name__)

class RecommendationEngine:
    """Main engine orchestrating the recommendation pipeline using Groq (Llama 4 Scout)."""

    def __init__(
        self,
        openrouter_service: Optional[OpenRouterService],
        groq_service: GroqService,
        cache_manager: CacheManager,
        clothing_image_service: Optional[ClothingImageService] = None,
        vton_service=None
    ):
        self.openrouter_service = openrouter_service
        self.groq_service = groq_service
        self.cache_manager = cache_manager
        self.clothing_image_service = clothing_image_service
        self.vton_service = vton_service

    def _clean_json_response(self, response_str: str) -> str:
        """
        Clean the response string to extract valid JSON.
        Removes markdown code blocks, extra text, and whitespace.
        """
        if not response_str or not response_str.strip():
            raise ValueError("Received empty response from API")

        # Remove leading/trailing whitespace
        response_str = response_str.strip()

        # Remove markdown code blocks
        if response_str.startswith("```json"):
            response_str = response_str[7:]
        elif response_str.startswith("```"):
            response_str = response_str[3:]

        if response_str.endswith("```"):
            response_str = response_str[:-3]

        response_str = response_str.strip()

        # Try to extract JSON object if there's extra text
        # Look for the first { and last }
        first_brace = response_str.find('{')
        last_brace = response_str.rfind('}')

        if first_brace != -1 and last_brace != -1 and last_brace > first_brace:
            response_str = response_str[first_brace:last_brace + 1]

        return response_str

    async def generate_recommendations(
        self,
        image: str,
        occasion: str,
        style: str,
        user_attributes: UserAttributes = None,
        include_brands: bool = False
    ) -> RecommendationResponse:
        """
        Simplified Workflow using Groq Llama:
```

```
        1. Analyze image and validate outfit appropriateness for occasion
        2. If inappropriate: Suggest 3 different outfit styles
        3. No visualization generation
        """
        total_start = time.time()

        try:
            # Analyze outfit appropriateness
            logger.info("Analyzing outfit with Groq (Llama 4 Vision)...")

            analysis_prompt = f"""Analyze this image and determine if the outfit is appropriate for the occasion: "
{occasion}".

Please provide your analysis in the following format:

APPROPRIATE: [Yes/No]
CRITIQUE: [Explain why the outfit is or isn't appropriate for {occasion}]
SUGGESTIONS: [If appropriate, suggest minor improvements. If not appropriate, explain what should be worn instead]

If the outfit is NOT appropriate, also provide 3 different outfit recommendations for {occasion}:

OUTFIT 1:
- Name: [Outfit style name]
- Description: [Brief description]
- Top: [Specific top item]
- Bottom: [Specific bottom item]
- Colors: [Color palette]
- Why it works: [Rationale]

OUTFIT 2:
[Same format]

OUTFIT 3:
[Same format]

Be specific and practical in your recommendations."""

            # Call Groq (Llama 4 Scout)
            response_text = await self.groq_service.analyze_image(image, analysis_prompt)
            logger.info(f"Groq Response (first 500 chars): {response_text[:500]}")

            # Parse the response
            is_appropriate = "APPROPRIATE: Yes" in response_text or "APPROPRIATE:Yes" in response_text

            # Extract critique
            critique = ""
            if "CRITIQUE:" in response_text:
                critique_start = response_text.find("CRITIQUE:") + len("CRITIQUE:")
                critique_end = response_text.find("SUGGESTIONS:", critique_start)
                if critique_end == -1:
                    critique_end = response_text.find("OUTFIT 1:", critique_start)
                if critique_end == -1:
                    critique_end = len(response_text)
                critique = response_text[critique_start:critique_end].strip()

            # Extract suggestions
            suggestions = ""
            if "SUGGESTIONS:" in response_text:
                sugg_start = response_text.find("SUGGESTIONS:") + len("SUGGESTIONS:")
                sugg_end = response_text.find("OUTFIT 1:", sugg_start)
                if sugg_end == -1:
                    sugg_end = len(response_text)
                suggestions = response_text[sugg_start:sugg_end].strip()

            # Build recommendations list
            recommendations = []
            generated_avatars = []

            if not is_appropriate:
                # Parse the 3 outfit recommendations
                for i in range(1, 4):
                    outfit_marker = f"OUTFIT {i}:"
                    if outfit_marker in response_text:
                        outfit_start = response_text.find(outfit_marker)
                        outfit_end = response_text.find(f"OUTFIT {i+1}:", outfit_start)
```

```
                        if outfit_end == -1:
                            outfit_end = len(response_text)

                        outfit_text = response_text[outfit_start:outfit_end]

                        # Extract fields
                        def extract_field(text, field_name):
                            if f"{field_name}:" in text:
                                start = text.find(f"{field_name}:") + len(f"{field_name}:")
                                # Find next field or end
                                next_fields = ["Name:", "Description:", "Top:", "Bottom:", "Colors:", "Why it works:",
"OUTFIT"]
                                end = len(text)
                                for nf in next_fields:
                                    pos = text.find(nf, start)
                                    if pos != -1 and pos < end:
                                        end = pos
                                return text[start:end].strip()
                            return ""

                        outfit_name = extract_field(outfit_text, "Name") or f"Outfit Option {i}"
                        description = extract_field(outfit_text, "Description") or "Recommended outfit"
                        top = extract_field(outfit_text, "Top") or "Appropriate top"
                        bottom = extract_field(outfit_text, "Bottom") or "Appropriate bottom"
                        colors_str = extract_field(outfit_text, "Colors") or "Neutral colors"
                        rationale = extract_field(outfit_text, "Why it works") or "Suitable for the occasion"

                        colors = [c.strip() for c in colors_str.split(",") if c.strip()]

                        rec = OutfitRecommendation(
                            outfit_name=outfit_name,
                            description=description,
                            top=top,
                            bottom=bottom,
                            shoes=None,
                            accessories=[],
                            colors=colors,
                            rationale=rationale,
                            styling_tips=[suggestions] if suggestions else [],
                            confidence_score=0.85,
                            why_it_works=rationale,
                            match_score=85
                        )
                        recommendations.append(rec)

                if recommendations and self.clothing_image_service:
                    recommendations = await self._attach_clothing_images(recommendations)

            # Build response
            processing_time = time.time() - total_start

            return RecommendationResponse(
                success=True,
                physical_analysis=None,  # Not needed in simplified workflow
                recommendations=recommendations,
                general_styling_tips=[suggestions] if suggestions else [],
                occasion=occasion,
                style=style,
                processing_time=processing_time,
                used_api="GROQ+LLAMA4",
                avatar_url=None,
                is_appropriate=is_appropriate,
                critique=critique,
                improvement_suggestions=suggestions,
                generated_avatars=generated_avatars  # Include generated outfit visualizations
            )

        except Exception as e:
            logger.error(f"Error in generate_recommendations: {str(e)}")
            import traceback
            logger.error(f"Traceback: {traceback.format_exc()}")

            # Return error response
            return RecommendationResponse(
                success=False,
```

```
                physical_analysis=None,
                recommendations=[],
                general_styling_tips=[],
                occasion=occasion,
                style=style,
                processing_time=time.time() - total_start,
                used_api="GROQ+LLAMA4",
                avatar_url=None,
                is_appropriate=False,
                critique=f"Error during analysis: {str(e)}",
                improvement_suggestions="Please try again or check the logs.",
                generated_avatars=[]
            )

    async def quick_analysis(self, image: str) -> Dict[str, Any]:
        """Quick analysis without full recommendations."""
        start_time = time.time()

        try:
            analysis_prompt = """Analyze the person in this image and provide a quick analysis.

CRITICAL: Respond with ONLY valid JSON. No markdown, no code blocks, no additional text.

Required JSON structure (ALL fields are mandatory):
{
    "body_type": "string (e.g., Rectangle, Hourglass, Pear, Apple, Inverted Triangle)",
    "skin_tone": "string (e.g., Fair, Light, Medium, Tan, Dark)",
    "proportions": "string (describe body proportions)",
    "skin_undertone": "string (e.g., Warm, Cool, Neutral)",
    "recommended_colors": ["color1", "color2", "color3"],
    "colors_to_avoid": ["color1", "color2"],
    "flattering_styles": ["style1", "style2", "style3"],
    "style_notes": "string (styling notes)"
}

Return ONLY the JSON object, nothing else."""

            if self.openrouter_service:
                analysis_json_str = await self.openrouter_service.get_image_description(image, analysis_prompt)
            else:
                analysis_json_str = await self.groq_service.analyze_image(image, analysis_prompt)

            # Log raw response for debugging
            logger.info(f"Raw quick analysis response (first 300 chars): {analysis_json_str[:300]}")

            # Clean the response
            try:
                cleaned_analysis = self._clean_json_response(analysis_json_str)
                analysis = json.loads(cleaned_analysis)
                logger.info("Successfully parsed quick analysis JSON")
            except json.JSONDecodeError as e:
                logger.error(f"Failed to parse quick analysis JSON. Raw response: {analysis_json_str[:500]}")
                raise ValueError(f"Invalid JSON from quick analysis: {str(e)}")

            return {
                "success": True,
                "body_type": analysis.get("body_type", "Unknown"),
                "skin_tone": analysis.get("skin_tone", "Unknown"),
                "recommended_colors": analysis.get("recommended_colors", []),
                "flattering_styles": analysis.get("flattering_styles", []),
                "processing_time": time.time() - start_time,
                "used_api": "openrouter" if self.openrouter_service else "groq"
            }

        except Exception as e:
            logger.error(f"Error in quick analysis: {str(e)}")
            raise

    async def _attach_clothing_images(
        self,
        recommendations: list[OutfitRecommendation],
    ) -> list[OutfitRecommendation]:
        enriched = []

        for rec in recommendations:
```

```
            try:
                images = await self.clothing_image_service.get_outfit_images(
                    top=rec.top,
                    bottom=rec.bottom,
                    shoes=rec.shoes or "",
                    accessories=rec.accessories,
                )
                rec.top_image_url = images.get("top_image_url")
                rec.bottom_image_url = images.get("bottom_image_url")
                rec.shoes_image_url = images.get("shoes_image_url")
            except Exception as image_error:
                logger.warning(f"Failed to enrich clothing images for '{rec.outfit_name}': {image_error}")

            enriched.append(rec)

        return enriched
```

backend/services/groq_service.py

```
import os
import logging
from groq import AsyncGroq

logger = logging.getLogger(__name__)

class GroqService:
    def __init__(self, api_key: str = None):
        api_key = api_key or os.getenv("GROQ_API_KEY")
        self.client = None
        if not api_key:
            logger.warning("GROQ_API_KEY not found in environment variables")
        else:
            self.client = AsyncGroq(api_key=api_key)
        self.model = "meta-llama/llama-4-scout-17b-16e-instruct" # Current Llama 4 Vision model
        logger.info(f"GroqService initialized with model: {self.model}")

    async def analyze_image(self, image_base64: str, prompt: str) -> str:
        """
        Analyze an image using Groq's Llama 3.2 Vision model.
        Returns the text response.
        """
        try:
            if not self.client:
                raise RuntimeError("GROQ_API_KEY is missing. Please set GROQ_API_KEY in your environment.")

            logger.info(f"Sending image analysis request to Groq ({self.model})...")

            # Ensure base64 string has prefix if missing (Groq expects data URL or just base64? SDK handles URL
usually)
            # The SDK expects a data URL for base64 images: ...
            if not image_base64.startswith("data:image"):
                # Assume jpeg if not specified, though frontend usually sends full data URL
                # If the backend receives raw base64, we add the prefix.
                # Based on previous code, it seems we might have raw base64 or data URL.
                # Let's ensure it's a data URL.
                image_url = f"{image_base64}"
            else:
                image_url = image_base64

            chat_completion = await self.client.chat.completions.create(
                messages=[
                    {
                        "role": "user",
                        "content": [
                            {"type": "text", "text": prompt},
                            {
                                "type": "image_url",
                                "image_url": {
                                    "url": image_url,
                                },
                            },
                        ],
                    }
                ],
                model=self.model,
                temperature=0.5,
                max_tokens=1024,
            )

            response_text = chat_completion.choices[0].message.content
            logger.info("Groq analysis complete.")
            return response_text

        except Exception as e:
            logger.error(f"Error calling Groq API: {str(e)}")
            raise e
```

backend/services/openrouter_service.py

```
import logging
from openai import AsyncOpenAI

logger = logging.getLogger(__name__)

class OpenRouterService:
    def __init__(self, api_key: str):
        self.api_key = api_key
        self.client = AsyncOpenAI(
            base_url="https://openrouter.ai/api/v1",
            api_key=api_key,
        )
        self.model = "meta-llama/llama-3.2-11b-vision-instruct"
        logger.info(f"OpenRouter initialized with model: {self.model}")

    async def describe_outfit(self, image_base64: str, prompt: str = None) -> str:
        """
        Simply describe what the person is wearing in the image.

        Args:
            image_base64: Base64 encoded image string
            prompt: Custom prompt (optional)

        Returns:
            Plain text description of the outfit
        """
        try:
            logger.info("Analyzing outfit...")

            if not prompt:
                prompt = "Describe what this person is wearing. Include clothing items, colors, and style. Keep it
simple and direct."

            response = await self.client.chat.completions.create(
                model=self.model,
                messages=[
                    {
                        "role": "user",
                        "content": [
                            {
                                "type": "text",
                                "text": prompt
                            },
                            {
                                "type": "image_url",
                                "image_url": {
                                    "url": f"{image_base64}"
                                }
                            }
                        ]
                    }
                ],
                max_tokens=500,
                temperature=0.5
            )

            description = response.choices[0].message.content
            logger.info(f"Successfully got outfit description")
            return description

        except Exception as e:
            logger.error(f"Error describing outfit: {str(e)}")
            raise e

    async def get_image_description(self, image_base64: str, prompt: str = None) -> str:
        """Alias for describe_outfit to maintain compatibility with RecommendationEngine."""
        return await self.describe_outfit(image_base64, prompt)
```

backend/services/gemini_vton_service.py

```
import base64
import binascii
import logging
import asyncio
import os
import ssl
import time
from typing import Any, Dict, Optional

import httpx
from google import genai
from google.genai import types

logger = logging.getLogger(__name__)


class GeminiVTONService:
    """VTON service backed by Gemini image preview model."""

    def __init__(self, api_key: str, model: str = "gemini-3-pro-image-preview"):
        if not api_key:
            raise ValueError("GEMINI_API_KEY is required for GeminiVTONService")

        self.api_key = api_key
        self.model = model

    @staticmethod
    def _decode_b64(image_base64: str) -> bytes:
        try:
            return base64.b64decode(image_base64)
        except (binascii.Error, ValueError) as exc:
            raise RuntimeError(f"Invalid base64 image payload: {exc}") from exc

    def _build_config(self) -> types.GenerateContentConfig:
        return types.GenerateContentConfig(
            response_modalities=["IMAGE", "TEXT"],
        )

    @staticmethod
    def _extract_chunk_parts(chunk: Any) -> list[Any]:
        parts = getattr(chunk, "parts", None)
        if parts:
            return list(parts)

        candidates = getattr(chunk, "candidates", None) or []
        for candidate in candidates:
            content = getattr(candidate, "content", None)
            candidate_parts = getattr(content, "parts", None)
            if candidate_parts:
                return list(candidate_parts)

        return []

    @staticmethod
    def _safe_chunk_text(chunk: Any) -> str:
        try:
            return getattr(chunk, "text", None) or ""
        except Exception:
            return ""

    @staticmethod
    def _is_transient_transport_error(exc: BaseException) -> bool:
        current: Optional[BaseException] = exc
        while current is not None:
            if isinstance(current, (ssl.SSLError, httpx.TransportError, ConnectionError, TimeoutError)):
                return True
            current = getattr(current, "__cause__", None) or getattr(current, "__context__", None)
        return False

    def _generate_content_with_retry(
        self,
        client: genai.Client,
```

```
        *,
        prompt: str,
        image_parts: list[types.Part],
        attempts: int = 3,
        base_delay_seconds: float = 1.0,
    ) -> Any:
        contents = [
            types.Content(
                role="user",
                parts=[types.Part.from_text(text=prompt), *image_parts],
            ),
        ]

        last_error: Optional[Exception] = None
        for attempt in range(1, attempts + 1):
            try:
                return client.models.generate_content(
                    model=self.model,
                    contents=contents,
                    config=self._build_config(),
                )
            except Exception as exc:
                last_error = exc
                if attempt >= attempts or not self._is_transient_transport_error(exc):
                    raise

                logger.warning(
                    "Transient Gemini transport failure on attempt %s/%s for model %s: %s",
                    attempt,
                    attempts,
                    self.model,
                    exc,
                )
                time.sleep(base_delay_seconds * attempt)

        if last_error is not None:
            raise last_error
        raise RuntimeError("Gemini request failed before a response was returned.")

    def _generate_image_sync(self, prompt: str, image_parts: list[types.Part]) -> str:
        # Prevent SDK from picking an unrelated GOOGLE_API_KEY from parent env.
        os.environ["GEMINI_API_KEY"] = self.api_key
        os.environ["GOOGLE_API_KEY"] = self.api_key
        client = genai.Client(api_key=self.api_key)
        response = self._generate_content_with_retry(
            client,
            prompt=prompt,
            image_parts=image_parts,
        )

        image_bytes = None
        streamed_text = []
        for part in self._extract_chunk_parts(response):
            if getattr(part, "inline_data", None) and getattr(part.inline_data, "data", None):
                image_bytes = part.inline_data.data
                break

        response_text = self._safe_chunk_text(response)
        if response_text:
            streamed_text.append(response_text)

        if image_bytes is None:
            text_hint = "".join(streamed_text).strip()
            raise RuntimeError(f"Gemini did not return an image. Model text: {text_hint[:500]}")

        return base64.b64encode(image_bytes).decode("utf-8")

    async def generate_outfit_on_person(
        self,
        original_image_base64: str,
        outfit_description: str,
        body_attributes: Optional[Dict[str, str]] = None,
        mime_type: str = "image/jpeg",
    ) -> str:
        body_type = (body_attributes or {}).get("body type", "average")
```

```
        skin_tone = (body_attributes or {}).get("skin tone", "medium")
        gender = (body_attributes or {}).get("gender", "person")

        prompt = (
            "Edit the provided person photo to keep the same person identity, face, pose, and framing. "
            f"Change only the outfit to: {outfit_description}. "
            f"Person details: {gender}, {body_type} build, {skin_tone} skin tone. "
            "Photorealistic fashion photography, clean details, realistic fabric, natural lighting. "
            "Do not add extra people, text, watermark, or logos."
        )

        image_parts = [
            types.Part.from_bytes(
                data=self._decode_b64(original_image_base64),
                mime_type=mime_type,
            )
        ]

        return await asyncio.to_thread(self._generate_image_sync, prompt, image_parts)

    async def generate_outfit_from_garment(
        self,
        original_image_base64: str,
        garment_image_base64: str,
        body_attributes: Optional[Dict[str, str]] = None,
        person_mime_type: str = "image/jpeg",
        garment_mime_type: str = "image/jpeg",
        extra_instruction: str = "",
    ) -> str:
        body_type = (body_attributes or {}).get("body type", "average")
        skin_tone = (body_attributes or {}).get("skin tone", "medium")
        gender = (body_attributes or {}).get("gender", "person")

        prompt = (
            "Use the first image as the person reference and the second image as the garment reference. "
            "Dress the same person from image one in the garment from image two. "
            "Keep person identity, face, body proportions, and pose consistent with image one. "
            f"Person details: {gender}, {body_type} build, {skin_tone} skin tone. "
            "Preserve garment design details from image two as closely as possible: cut, fabric texture, color, and
silhouette. "
            "Photorealistic fashion output, clean lighting, no text, no watermark, no extra people. "
            f"Additional instruction: {extra_instruction or 'none'}"
        )

        image_parts = [
            types.Part.from_bytes(
                data=self._decode_b64(original_image_base64),
                mime_type=person_mime_type,
            ),
            types.Part.from_bytes(
                data=self._decode_b64(garment_image_base64),
                mime_type=garment_mime_type,
            ),
        ]

        return await asyncio.to_thread(self._generate_image_sync, prompt, image_parts)

    async def generate_avatar(
        self,
        prompt: str,
        width: int = 768,
        height: int = 1024,
        **_: Any,
    ) -> str:
        # Width and height are accepted for API compatibility with previous service.
        rich_prompt = (
            f"{prompt}. Render as a photorealistic full-body fashion image. "
            f"Target aspect ratio close to {width}:{height}."
        )
        return await asyncio.to_thread(self._generate_image_sync, rich_prompt, [])

    async def generate_outfit_visualization(
        self,
        outfit_description: str,
        body_type: str = "average",
```

```
        skin_tone: str = "medium",
        style: str = "modern",
    ) -> str:
        prompt = (
            f"A {style} style full-body fashion portrait of a {body_type} build person with {skin_tone} skin tone, "
            f"wearing {outfit_description}. High quality studio fashion photography."
        )
        return await self.generate_avatar(prompt=prompt)
```

backend/services/clothing_image_service.py

```
"""
Clothing Image Service - Fetches real clothing images from the web.
Uses duckduckgo_search library for actual image results with Bing scrape fallback.
No API keys required.
"""

import asyncio
import logging
import urllib.parse
from typing import Dict, List, Optional

logger = logging.getLogger(__name__)


class ClothingImageService:
    """
    Fetches real clothing product images from the web using DuckDuckGo Image Search.
    No API keys required.
    """

    def __init__(self):
        self._cache: Dict[str, Optional[str]] = {}

    def _normalize_description(self, description: str) -> str:
        cleaned = (description or "").strip()
        if not cleaned:
            return ""

        for old, new in ((" and ", " "), (" or ", " "), ("/", " ")):
            cleaned = cleaned.replace(old, new)

        return " ".join(cleaned.split())

    def _build_query(self, description: str, category: str) -> str:
        """Build a search query that favors product-only item photos."""
        filler = {"a", "an", "the", "or", "with", "for", "e.g.", "etc", "and", "of", "in", "very"}
        description = self._normalize_description(description)
        words = [w.strip("*-•") for w in description.split() if w.lower().strip("*-•") not in filler]
        base = " ".join(words)

        suffix_map = {
            "top": "topwear product photo flat lay catalog isolated ecommerce no model no person",
            "bottom": "pants trousers bottomwear product photo flat lay catalog isolated ecommerce no model no
person",
            "shoes": "footwear shoes product photo isolated ecommerce no model no person",
        }
        suffix = suffix_map.get(category, "clothing product photo flat lay catalog isolated ecommerce no model no
person")
        return f"{base} {suffix}"

    def _looks_like_product_image(self, url: str, source: str = "", title: str = "") -> bool:
        haystack = " ".join([url or "", source or "", title or ""]).lower()

        rejected = [
            "pinterest",
            "youtube",
            "instagram",
            "facebook",
            "twitter",
            "reddit",
            "tumblr",
            "lookbook",
            "blog",
            "editorial",
            "celebrity",
            "streetstyle",
            "runway",
            "polyvore",
        ]
        if any(term in haystack for term in rejected):
            return False
```

```
        preferred = [
            "product",
            "catalog",
            "ecommerce",
            "shop",
            "store",
            "isolated",
            "flat",
            "lay",
            "packshot",
            "transparent",
            "white-background",
        ]
        discouraged = ["model", "person", "man", "woman", "girl", "boy", "wearing"]

        if any(term in haystack for term in preferred):
            return True
        if any(term in haystack for term in discouraged):
            return False
        return True

    async def _search_ddg_images(self, query: str) -> Optional[str]:
        """Search DuckDuckGo Images using the duckduckgo_search library."""
        try:
            import warnings

            with warnings.catch_warnings():
                warnings.simplefilter("ignore")
                from duckduckgo_search import DDGS

            def _sync_search():
                with DDGS() as ddgs:
                    return list(
                        ddgs.images(
                            keywords=query,
                            max_results=15,
                            safesearch="moderate",
                        )
                    )

            results = await asyncio.to_thread(_sync_search)

            if results:
                tier1_sites = [
                    "amazon",
                    "nordstrom",
                    "zara",
                    "hm.com",
                    "uniqlo",
                    "asos",
                    "macys",
                    "nike",
                    "adidas",
                    "forever21",
                    "gap",
                    "oldnavy",
                    "target",
                    "walmart",
                ]
                tier2_sites = [
                    "myntra",
                    "flipkart",
                    "ajio",
                    "shopify",
                    "etsy",
                    "zalando",
                    "boohoo",
                    "prettylittlething",
                    "shein",
                ]

                for result in results:
                    url = result.get("image", "")
                    source = result.get("source", "").lower()
                    title = result.get("title", "")
```

```
                    if any(site in url.lower() or site in source for site in tier1_sites) and
self._looks_like_product_image(url, source, title):
                        logger.info(f"DDG tier1 image: {url[:80]}")
                        return url

                for result in results:
                    url = result.get("image", "")
                    source = result.get("source", "").lower()
                    title = result.get("title", "")
                    if any(site in url.lower() or site in source for site in tier2_sites) and
self._looks_like_product_image(url, source, title):
                        logger.info(f"DDG tier2 image: {url[:80]}")
                        return url

                for result in results:
                    url = result.get("image", "")
                    source = result.get("source", "")
                    title = result.get("title", "")
                    if url and self._looks_like_product_image(url, source, title):
                        logger.info(f"DDG filtered image: {url[:80]}")
                        return url

        except ImportError:
            logger.warning("duckduckgo_search not installed - pip install duckduckgo_search")
        except Exception as e:
            logger.warning(f"DDG image search failed for '{query}': {e}")

        return None

    async def _search_bing_scrape(self, query: str) -> Optional[str]:
        """Scrape Bing Images for a result (no API key needed) with shopping filter."""
        import httpx
        import re as _re

        try:
            encoded = urllib.parse.quote_plus(query)
            url = f"https://www.bing.com/images/search?q={encoded}&qft=+filterui:photo-photo+filterui:imagesize-
large&first=1"
            headers = {
                "User-Agent": (
                    "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
                    "AppleWebKit/537.36 (KHTML, like Gecko) "
                    "Chrome/120.0.0.0 Safari/537.36"
                ),
                "Accept": "text/html",
                "Accept-Language": "en-US,en;q=0.9",
            }
            async with httpx.AsyncClient(timeout=12.0, follow_redirects=True) as client:
                resp = await client.get(url, headers=headers)
                if resp.status_code == 200:
                    matches = _re.findall(r'murl&quot;:&quot;(https?://[^&]+?)&quot;', resp.text)
                    if matches:
                        for match in matches[:10]:
                            if self._looks_like_product_image(match):
                                logger.info(f"Bing scrape image: {match[:80]}")
                                return match

                        logger.info(f"Bing scrape (fallback) image: {matches[0][:80]}")
                        return matches[0]

        except Exception as e:
            logger.warning(f"Bing scrape failed for '{query}': {e}")

        return None

    async def search_clothing_image(
        self,
        item_description: str,
        category: str = "clothing",
    ) -> Optional[str]:
        """Search for a single clothing item image. Returns URL or None."""
        query = self._build_query(item_description, category)

        if query in self._cache:
            return self._cache[query]
```

```
        url = await self._search_ddg_images(query)
        if not url:
            url = await self._search_bing_scrape(query)

        self._cache[query] = url
        return url

    async def get_outfit_images(
        self,
        top: str,
        bottom: str,
        shoes: str,
        accessories: List[str] = None,
    ) -> Dict[str, Optional[str]]:
        """
        Fetch images for a complete outfit concurrently.
        Returns dict: top_image_url, bottom_image_url, shoes_image_url
        """
        logger.info(f"Fetching outfit images - top='{top[:40]}', bottom='{bottom[:40]}', shoes='{shoes[:40]}'")

        tasks = [
            self.search_clothing_image(top, "top"),
            self.search_clothing_image(bottom, "bottom"),
            self.search_clothing_image(shoes, "shoes"),
        ]
        results = await asyncio.gather(*tasks, return_exceptions=True)

        outfit = {
            "top_image_url": results[0] if not isinstance(results[0], (Exception, type(None))) else None,
            "bottom_image_url": results[1] if not isinstance(results[1], (Exception, type(None))) else None,
            "shoes_image_url": results[2] if not isinstance(results[2], (Exception, type(None))) else None,
        }

        for key, val in outfit.items():
            if val:
                logger.info(f"  ok {key}: {val[:60]}...")
            else:
                logger.warning(f"  missing {key}: no image found")

        return outfit
```

frontend/pages/index.tsx

```
import { useRef } from 'react'
import Head from 'next/head'
import Link from 'next/link'
import { motion } from 'framer-motion'
import GlassCube from '../components/GlassCube'
import CameraReelPlaceholder from '../components/CameraReelPlaceholder'

export default function Home() {
  const manifestoRef = useRef<HTMLElement>(null)
  const intelligenceRef = useRef<HTMLElement>(null)
  const syndicateRef = useRef<HTMLElement>(null)
  const canvasRef = useRef<HTMLDivElement>(null)

  const scrollTo = (section: 'manifesto' | 'intelligence' | 'syndicate') => {
    if (section === 'manifesto') manifestoRef.current?.scrollIntoView({ behavior: 'smooth' })
    if (section === 'intelligence') intelligenceRef.current?.scrollIntoView({ behavior: 'smooth' })
    if (section === 'syndicate') syndicateRef.current?.scrollIntoView({ behavior: 'smooth' })
  }

  const dossierCards = [
    {
      id: '091',
      title: 'Ghostly Textures',
      image: 'https://images.unsplash.com/photo-1515886657613-9f3515b0c78f?auto=format&fit=crop&w=1200&q=80'
    },
    {
      id: '092',
      title: 'Void Aesthetics',
      image: 'https://images.unsplash.com/photo-1509631179647-0177331693ae?auto=format&fit=crop&w=1200&q=80'
    },
    {
      id: '093',
      title: 'Lunar Minimalism',
      image: 'https://images.unsplash.com/photo-1485230895905-ec40ba36b9bc?auto=format&fit=crop&w=1200&q=80'
    }
  ]

  const handleNav = (section: 'manifesto' | 'intelligence' | 'syndicate') => {
    scrollTo(section)
  }

  const handleCanvasMove = (event: React.MouseEvent<HTMLDivElement>) => {
    const rect = event.currentTarget.getBoundingClientRect()
    const x = event.clientX - rect.left
    const y = event.clientY - rect.top
    canvasRef.current?.style.setProperty('--mx', `${x}px`)
    canvasRef.current?.style.setProperty('--my', `${y}px`)
  }

  const resetCanvasGlow = () => {
    canvasRef.current?.style.setProperty('--mx', '50%')
    canvasRef.current?.style.setProperty('--my', '35%')
  }

  return (
    <>
      <Head>
        <title>ATRIQUET | Style Intelligence Atelier</title>
        <meta name="description" content="Algorithmic style intelligence in a brutalist editorial interface." />
        <meta name="viewport" content="width=device-width, initial-scale=1" />
      </Head>

      <div
        ref={canvasRef}
        className="interactive-canvas min-h-screen text-white"
        onMouseMove={handleCanvasMove}
        onMouseLeave={resetCanvasGlow}
      >
        <nav className="fixed top-0 left-0 z-50 w-full border-b border-white/20 bg-black/45 backdrop-blur-md">
          <div className="mx-auto flex max-w-[1600px] items-center justify-between px-6 py-5 md:px-12">
            <button
              onClick={() => handleNav('manifesto')}
```

```
              className="font-mono text-[10px] uppercase tracking-[0.34em]"
            >
              ATRIQUET // 0.0
            </button>
            <div className="flex items-center gap-4 md:gap-10 font-mono text-[10px] uppercase tracking-[0.3em]">
              <Link href="/homepage" className="hover:text-[#ff0000]">Homepage</Link>
              <button onClick={() => handleNav('intelligence')} className="hover:text-[#ff0000]">Archive</button>
              <button onClick={() => handleNav('intelligence')} className="hover:text-[#ff0000]">Intelligence</button>
              <button onClick={() => handleNav('syndicate')} className="hover:text-[#ff0000]">Syndicate</button>
              <Link href="/vton" className="hover:text-[#ff0000]">VTON</Link>
            </div>
          </div>
        </nav>

        <main className="relative z-[1] pt-20">
          <section ref={manifestoRef} className="atelier-grid relative min-h-screen overflow-hidden border-b border-
white/20 px-6 py-10 md:px-12">
              <div className="mx-auto grid h-full max-w-[1600px] grid-cols-1 gap-8 lg:grid-cols-12">
                <div className="col-span-12 flex flex-col justify-center lg:col-span-8">
                  <motion.h1
                    initial={{ opacity: 0, y: 28 }}
                    animate={{ opacity: 1, y: 0 }}
                    transition={{ duration: 0.8 }}
                    className="font-display text-[min(24vw,180px)] italic leading-[0.83]"
                  >
                    ATRIQUET
                  </motion.h1>
                  <motion.p
                    initial={{ opacity: 0, y: 20 }}
                    animate={{ opacity: 1, y: 0 }}
                    transition={{ duration: 0.8, delay: 0.15 }}
                    className="mt-8 max-w-xl font-mono text-[10px] uppercase leading-6 tracking-[0.28em] text-white/70
md:text-xs"
                  >
                    Algorithmic style intelligence for luxury buyers and creative directors. No fluff. No trend
theater. Only raw, uncompromising signal.
                  </motion.p>
                </div>
                <motion.div
                  initial={{ opacity: 0, y: 20 }}
                  animate={{ opacity: 1, y: 0 }}
                  transition={{ duration: 0.8, delay: 0.3 }}
                  className="col-span-12 flex items-end justify-start lg:col-span-4 lg:justify-end"
                >
                  <button
                    onClick={() => scrollTo('intelligence')}
                    className="brutal-btn h-20 w-full max-w-[320px] font-heading text-xs uppercase tracking-[0.3em]
md:w-[320px]"
                  >
                    Enter The Archive
                  </button>
                </motion.div>
              </div>

              <motion.div
                initial={{ opacity: 0, y: 28 }}
                animate={{ opacity: 1, y: 0 }}
                transition={{ duration: 0.8, delay: 0.35 }}
                className="mx-auto mt-10 w-full max-w-[1600px] lg:mt-14"
              >
                <CameraReelPlaceholder />
              </motion.div>

              <div className="absolute bottom-8 left-6 hidden items-end gap-4 md:flex md:left-12">
                <div className="h-28 w-px bg-white/60" />
                <span className="font-mono text-[10px] uppercase tracking-[0.26em] text-white/80 [writing-
mode:vertical-rl]">
                  Coordinates: 48.8566 N, 2.3522 E
                </span>
              </div>
          </section>

          <section ref={intelligenceRef} className="border-b border-white/20 px-6 py-20 md:px-12">
              <div className="mx-auto max-w-[1600px]">
                <div className="mb-12 flex items-end justify-between border-b border-white/20 pb-5">
```

```
                <h2 className="font-heading text-3xl uppercase tracking-[0.1em] md:text-5xl">The Intelligence</h2>
                <span className="font-mono text-[10px] uppercase tracking-[0.28em] text-white/80">[01]
Archive</span>
              </div>

              <div className="grid grid-cols-1 gap-2 md:grid-cols-2 xl:grid-cols-3">
                {dossierCards.map((card, index) => (
                  <motion.article
                    key={card.id}
                    initial={{ opacity: 0, y: 20 }}
                    whileInView={{ opacity: 1, y: 0 }}
                    viewport={{ once: true, amount: 0.2 }}
                    transition={{ duration: 0.5, delay: index * 0.1 }}
                    className="group relative h-[560px] cursor-pointer overflow-hidden border border-white/30"
                  >
                    <img
                      alt={card.title}
                      src={card.image}
                      className="h-full w-full object-cover grayscale transition-none group-hover:grayscale-0"
                    />
                    <div className="absolute inset-0 bg-black/30 group-hover:bg-black/0" />
                    <div className="absolute left-6 top-6 z-10 font-mono text-[10px] uppercase tracking-[0.26em]
text-white mix-blend-difference">
                      [ Report {card.id} ]
                    </div>
                    <h3 className="absolute bottom-16 left-6 z-10 bg-white px-3 py-2 font-display text-4xl italic
leading-tight text-black group-hover:bg-black group-hover:text-white md:text-5xl">
                      {card.title}
                    </h3>
                    <div className="absolute bottom-6 left-6 right-6 z-10 flex items-center justify-between font-
mono text-[10px] uppercase tracking-[0.26em] text-white mix-blend-difference">
                      <span>Archive // 2026</span>
                      <span>Open Dossier</span>
                    </div>
                  </motion.article>
                ))}
              </div>
            </div>
          </section>

          <section ref={syndicateRef} className="px-6 py-20 md:px-12 md:py-24 flex flex-col items-center">
            <div className="text-center mb-6">
              <p className="font-mono text-[10px] uppercase tracking-[0.28em] text-[#00E5FF]">The Syndicate</p>
              <h2 className="mt-4 font-display text-5xl italic leading-[0.9] md:text-7xl">Application For
Clearance</h2>
              <p className="mt-6 max-w-lg mx-auto font-mono text-[10px] uppercase tracking-widest leading-relaxed
text-white/50">
                Submit your profile for algorithmic curation. This gate remains narrow by design.
              </p>
            </div>

            {/* Glass Cube Monolith replaces the legacy form and controls its own flow natively */}
            <GlassCube />
          </section>

        </main>

        <div className="ticker-wrap fixed bottom-0 left-0 z-50 w-full border-t border-white/20 bg-black/70">
          <div className="ticker-track py-2 font-mono text-[10px] uppercase tracking-[0.27em] text-white/80">
            <span>Trend Velocity: +14.2%</span>
            <span>Current Sector: Neo-Monolith</span>
            <span>Atmospheric Density: 0.003kg/m3</span>
            <span>Signal Strength: Nominal</span>
            <span>Trend Velocity: +14.2%</span>
            <span>Current Sector: Neo-Monolith</span>
            <span>Atmospheric Density: 0.003kg/m3</span>
            <span>Signal Strength: Nominal</span>
          </div>
        </div>
      </div>
    </>
  )
}
```

frontend/pages/homepage.tsx

```
import Head from 'next/head'
import Link from 'next/link'
import { useRouter } from 'next/router'
import ScrollFrameSequence from '../components/ScrollFrameSequence'

export default function HomepageExperience() {
  const router = useRouter()

  return (
    <>
      <Head>
        <title>ATRIQUET | Homepage Experience</title>
        <meta
          name="description"
          content="Scroll-controlled wardrobe opening sequence that reveals the project title frame by frame."
        />
        <meta name="viewport" content="width=device-width, initial-scale=1" />
      </Head>

      <div className="min-h-screen bg-black text-white">
        <nav className="fixed left-0 top-0 z-50 w-full border-b border-white/20 bg-black/45 backdrop-blur-md">
          <div className="mx-auto flex max-w-[1600px] items-center justify-between px-6 py-5 md:px-12">
            <Link href="/" className="font-mono text-[10px] uppercase tracking-[0.34em]">
              ATRIQUET // 0.0
            </Link>
            <div className="flex items-center gap-4 font-mono text-[10px] uppercase tracking-[0.3em] md:gap-10">
              <span className="text-white/70">Homepage</span>
              <Link href="/" className="hover:text-[#ff0000]">
                Main Page
              </Link>
            </div>
          </div>
        </nav>

        <main className="pt-20">
          <ScrollFrameSequence
            frameCount={192}
            eyebrow="Wardrobe Opening Sequence"
            title="ATRIQUET"
            subtitle="Every scroll step advances the extracted frame sequence so the door opens gradually and reveals
the project title with direct user control."
            ctaLabel="Continue"
            onCtaClick={() => router.push('/')}
          />

          <section className="border-t border-white/15 bg-black px-6 py-20 md:px-12">
            <div className="mx-auto flex max-w-[1200px] flex-col items-start justify-between gap-8 md:flex-row
md:items-end">
              <div className="max-w-2xl">
                <p className="font-mono text-[10px] uppercase tracking-[0.3em] text-white/60">Sequence Complete</p>
                <h2 className="mt-4 font-display text-4xl italic md:text-6xl">The doors are open. Continue into the
main experience.</h2>
              </div>
              <button
                onClick={() => router.push('/')}
                className="border border-white bg-white px-8 py-4 font-heading text-xs uppercase tracking-[0.28em]
text-black transition hover:border-[#00E5FF] hover:bg-[#00E5FF] hover:text-white"
              >
                Continue
              </button>
            </div>
          </section>
        </main>
      </div>
    </>
  )
}
```

frontend/pages/vton.tsx

```
import { useCallback, useRef, useState } from 'react'
import Head from 'next/head'
import Link from 'next/link'
import axios from 'axios'
import { useDropzone } from 'react-dropzone'
import { AnimatePresence, motion } from 'framer-motion'

const API_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8000'

type VtonResponse = {
  success: boolean
  image?: string
  message?: string
}

function normalizeGeneratedImageSrc(value?: string): string | null {
  if (typeof value !== 'string') {
    return null
  }

  const trimmed = value.trim().replace(/^"+|"+$/g, '')
  if (!trimmed) {
    return null
  }

  const unwrapNestedBase64 = (payload: string) => {
    let current = payload

    for (let i = 0; i < 2; i += 1) {
      try {
        const decoded = atob(current)
        const looksLikeBase64 = /^[A-Za-z0-9+/=\r\n]+$/.test(decoded)
        if (looksLikeBase64) {
          current = decoded.trim().replace(/^"+|"+$/g, '')
          continue
        }
      } catch {
        return current
      }

      return current
    }

    return current
  }

  const detectMime = (payload: string) => {
    if (payload.startsWith('/9j/')) return 'image/jpeg'
    if (payload.startsWith('iVBORw0KGgo')) return 'image/png'
    if (payload.startsWith('R0lGOD')) return 'image/gif'
    if (payload.startsWith('UklGR')) return 'image/webp'
    return 'image/png'
  }

  if (trimmed.startsWith('data:image/')) {
    const parts = trimmed.split(',', 2)
    if (parts.length === 2) {
      const normalizedPayload = unwrapNestedBase64(parts[1])
      const mime = detectMime(normalizedPayload)
      return `data:${mime}base64,${normalizedPayload}`
    }
    return trimmed
  }

  if (trimmed.startsWith('http://') || trimmed.startsWith('https://')) {
    return trimmed
  }

  const normalizedPayload = unwrapNestedBase64(trimmed)
  const mime = detectMime(normalizedPayload)
  return `data:${mime}base64,${normalizedPayload}`
}
```

```
export default function VTONPage() {
  const canvasRef = useRef<HTMLDivElement>(null)
  const resultRef = useRef<HTMLElement>(null)

  const [image, setImage] = useState<File | null>(null)
  const [preview, setPreview] = useState<string | null>(null)
  const [garmentImage, setGarmentImage] = useState<File | null>(null)
  const [garmentPreview, setGarmentPreview] = useState<string | null>(null)
  const [outfitDescription, setOutfitDescription] = useState('Use the garment naturally fitted to the person and keep
realistic fabric folds.')
  const [bodyType, setBodyType] = useState('average')
  const [skinTone, setSkinTone] = useState('medium')
  const [gender, setGender] = useState('person')
  const [loading, setLoading] = useState(false)
  const [error, setError] = useState<string | null>(null)
  const [resultImage, setResultImage] = useState<string | null>(null)

  const onDropPerson = useCallback((acceptedFiles: File[]) => {
    const file = acceptedFiles[0]
    if (file) {
      setImage(file)
      setPreview(URL.createObjectURL(file))
      setError(null)
    }
  }, [])

  const onDropGarment = useCallback((acceptedFiles: File[]) => {
    const file = acceptedFiles[0]
    if (file) {
      setGarmentImage(file)
      setGarmentPreview(URL.createObjectURL(file))
      setError(null)
    }
  }, [])

  const { getRootProps: getPersonRootProps, getInputProps: getPersonInputProps, isDragActive: isPersonDragActive } =
useDropzone({
    onDrop: onDropPerson,
    accept: {
      'image/*': ['.jpeg', '.jpg', '.png']
    },
    maxFiles: 1,
    maxSize: 10 * 1024 * 1024
  })

  const { getRootProps: getGarmentRootProps, getInputProps: getGarmentInputProps, isDragActive: isGarmentDragActive }
= useDropzone({
    onDrop: onDropGarment,
    accept: {
      'image/*': ['.jpeg', '.jpg', '.png']
    },
    maxFiles: 1,
    maxSize: 10 * 1024 * 1024
  })

  const handleCanvasMove = (event: React.MouseEvent<HTMLDivElement>) => {
    const rect = event.currentTarget.getBoundingClientRect()
    const x = event.clientX - rect.left
    const y = event.clientY - rect.top
    canvasRef.current?.style.setProperty('--mx', `${x}px`)
    canvasRef.current?.style.setProperty('--my', `${y}px`)
  }

  const resetCanvasGlow = () => {
    canvasRef.current?.style.setProperty('--mx', '50%')
    canvasRef.current?.style.setProperty('--my', '35%')
  }

  const handleSubmit = async () => {
    if (!image) {
      setError('Please upload a person image first')
      return
    }
```

```
    if (!garmentImage) {
      setError('Please upload a garment image')
      return
    }

    setLoading(true)
    setError(null)

    try {
      const formData = new FormData()
      formData.append('image', image)
      formData.append('garment_image', garmentImage)
      formData.append('outfit_description', outfitDescription)
      formData.append('body_type', bodyType)
      formData.append('skin_tone', skinTone)
      formData.append('gender', gender)

      const response = await axios.post<VtonResponse>(`${API_URL}/api/generate-outfit-on-person`, formData, {
        headers: {
          'Content-Type': 'multipart/form-data'
        },
        timeout: 300000
      })

      const generatedImage = normalizeGeneratedImageSrc(response.data?.image)
      if (!generatedImage) {
        throw new Error('No usable image returned by the server')
      }

      setResultImage(generatedImage)

      setTimeout(() => {
        resultRef.current?.scrollIntoView({ behavior: 'smooth', block: 'start' })
      }, 120)
    } catch (err: any) {
      setError(err.response?.data?.detail || err.message || 'Failed to generate VTON output')
    } finally {
      setLoading(false)
    }
  }

  return (
    <>
      <Head>
        <title>ATRIQUET | VTON Atelier</title>
        <meta name="description" content="Virtual try-on powered by Gemini image preview model." />
        <meta name="viewport" content="width=device-width, initial-scale=1" />
      </Head>

      <div
        ref={canvasRef}
        className="interactive-canvas min-h-screen text-white"
        onMouseMove={handleCanvasMove}
        onMouseLeave={resetCanvasGlow}
      >
        <nav className="fixed top-0 left-0 z-50 w-full border-b border-white/20 bg-black/45 backdrop-blur-md">
          <div className="mx-auto flex max-w-[1600px] items-center justify-between px-6 py-5 md:px-12">
            <Link href="/" className="font-mono text-[10px] uppercase tracking-[0.34em]">ATRIQUET // 0.0</Link>
            <div className="flex items-center gap-4 md:gap-10 font-mono text-[10px] uppercase tracking-[0.3em]">
              <Link href="/" className="hover:text-[#ff0000]">Manifesto</Link>
              <span className="text-white/70">VTON LAB</span>
            </div>
          </div>
        </nav>

        <main className="relative z-[1] px-6 pb-24 pt-28 md:px-12">
          <section className="mx-auto max-w-[1600px] border border-white/20">
            <div className="grid grid-cols-1 lg:grid-cols-2">
              <div className="bg-black p-8 md:p-14">
                <p className="font-mono text-[10px] uppercase tracking-[0.28em] text-white/70">The VTON Dossier</p>
                <h1 className="mt-6 font-display text-5xl italic leading-[0.9] md:text-7xl">Virtual Try-On
Clearance</h1>
                <p className="mt-8 max-w-md font-body text-base leading-relaxed text-white/80">
                  Submit one portrait and one outfit directive. The lab keeps identity, pose, and frame while
replacing garment composition.
```

```
                </p>
                <div className="mt-12 h-px w-44 bg-white/30" />
                {preview && (
                  <div className="mt-10 border border-white/20 p-4">
                    <p className="mb-3 font-mono text-[10px] uppercase tracking-[0.22em] text-white/70">Source
Portrait</p>
                    <img src={preview} alt="Source" className="h-[360px] w-full object-cover" />
                  </div>
                )}
                {garmentPreview && (
                  <div className="mt-6 border border-white/20 p-4">
                    <p className="mb-3 font-mono text-[10px] uppercase tracking-[0.22em] text-white/70">Garment
Reference</p>
                    <img src={garmentPreview} alt="Garment" className="h-[360px] w-full object-cover" />
                  </div>
                )}
              </div>

              <div className="bg-white p-8 text-black md:p-14">
                <div className="space-y-7">
                  <div>
                    <label className="mb-3 block font-mono text-[10px] uppercase tracking-[0.25em]">Upload Person
Image</label>
                    <div
                      {...getPersonRootProps()}
                      className={`cursor-pointer border border-black p-8 text-center transition ${
                        isPersonDragActive ? 'bg-black text-white' : 'bg-white text-black'
                      }`}
                    >
                      <input {...getPersonInputProps()} />
                      <p className="font-heading text-lg uppercase tracking-[0.08em]">
                        {preview ? 'Replace portrait' : 'Drop portrait or click'}
                      </p>
                      <p className="mt-2 font-mono text-[10px] uppercase tracking-[0.2em] text-[#666]">JPG / PNG up to
10MB</p>
                    </div>
                  </div>

                  <div>
                    <label className="mb-3 block font-mono text-[10px] uppercase tracking-[0.25em]">Upload Garment
Image</label>
                    <div
                      {...getGarmentRootProps()}
                      className={`cursor-pointer border border-black p-8 text-center transition ${
                        isGarmentDragActive ? 'bg-black text-white' : 'bg-white text-black'
                      }`}
                    >
                      <input {...getGarmentInputProps()} />
                      <p className="font-heading text-lg uppercase tracking-[0.08em]">
                        {garmentPreview ? 'Replace garment reference' : 'Drop garment image or click'}
                      </p>
                      <p className="mt-2 font-mono text-[10px] uppercase tracking-[0.2em] text-[#666]">JPG / PNG up to
10MB</p>
                    </div>
                  </div>

                  <div>
                    <label className="mb-3 block font-mono text-[10px] uppercase tracking-[0.25em]">Outfit
Directive</label>
                    <textarea
                      value={outfitDescription}
                      onChange={(e) => setOutfitDescription(e.target.value)}
                      rows={5}
                      className="w-full border border-black p-4 font-body text-sm leading-relaxed outline-none
focus:bg-black focus:text-white"
                    />
                  </div>

                  <div className="grid grid-cols-1 gap-4 md:grid-cols-3">
                    <div>
                      <label className="mb-2 block font-mono text-[10px] uppercase tracking-[0.2em]">Body Type</label>
                      <input
                        value={bodyType}
                        onChange={(e) => setBodyType(e.target.value)}
                        className="w-full border border-black p-3 font-body text-sm outline-none focus:bg-black
```

```
focus:text-white"
                      />
                    </div>
                    <div>
                      <label className="mb-2 block font-mono text-[10px] uppercase tracking-[0.2em]">Skin Tone</label>
                      <input
                        value={skinTone}
                        onChange={(e) => setSkinTone(e.target.value)}
                        className="w-full border border-black p-3 font-body text-sm outline-none focus:bg-black
focus:text-white"
                      />
                    </div>
                    <div>
                      <label className="mb-2 block font-mono text-[10px] uppercase tracking-[0.2em]">Gender</label>
                      <input
                        value={gender}
                        onChange={(e) => setGender(e.target.value)}
                        className="w-full border border-black p-3 font-body text-sm outline-none focus:bg-black
focus:text-white"
                      />
                    </div>
                  </div>

                  <button
                    onClick={handleSubmit}
                    disabled={loading}
                    className="w-full border border-black bg-black px-8 py-5 font-heading text-xs uppercase tracking-
[0.3em] text-white transition hover:bg-[#ff0000] disabled:cursor-not-allowed disabled:opacity-60"
                  >
                    {loading ? 'Rendering VTON...' : 'Generate VTON'}
                  </button>

                  {error && (
                    <div className="border border-[#ff0000] bg-[#fff4f4] p-4 font-body text-sm text-[#b00000]">{error}
</div>
                  )}
                </div>
              </div>
            </div>
          </section>

          <AnimatePresence>
            {resultImage && (
              <motion.section
                ref={resultRef}
                initial={{ opacity: 0, y: -70, height: 0 }}
                animate={{ opacity: 1, y: 0, height: 'auto' }}
                exit={{ opacity: 0, y: -50, height: 0 }}
                transition={{
                  height: { duration: 0.55, ease: [0.22, 1, 0.36, 1] },
                  opacity: { duration: 0.35 },
                  y: { type: 'spring', stiffness: 90, damping: 20 }
                }}
                className="overflow-hidden"
              >
                <div className="mx-auto mt-6 max-w-[1600px] border border-white/20 bg-white p-6 text-black md:p-12">
                  <div className="mb-8 flex items-center justify-between border-b border-black/20 pb-5">
                    <h2 className="font-heading text-3xl uppercase tracking-[0.12em] md:text-4xl">VTON Result</h2>
                    <button
                      onClick={() => setResultImage(null)}
                      className="border border-black px-6 py-3 font-mono text-[10px] uppercase tracking-[0.24em]
transition hover:bg-black hover:text-white"
                    >
                      Close Result
                    </button>
                  </div>
                  <img
                    src={resultImage}
                    alt="Generated VTON output"
                    className="max-h-[900px] w-full object-contain"
                    onError={() => {
                      setResultImage(null)
                      setError('Generated image could not be displayed. Please try again with a different garment
photo.')
                    }}
```

```
                  />
                </div>
              </motion.section>
            )}
          </AnimatePresence>
        </main>
      </div>
    </>
  )
}
```

frontend/components/CameraReelPlaceholder.tsx

```
type ReelImage = {
  src: string
  title: string
}

const REEL_IMAGES: ReelImage[] = [
  { src: '/assets/fashion-reel/look-1.jpg', title: 'Monochrome Tailoring' },
  { src: '/assets/fashion-reel/look-2.jpg', title: 'Street Atelier Layering' },
  { src: '/assets/fashion-reel/look-3.jpg', title: 'Luxe Night Editorial' },
  { src: '/assets/fashion-reel/look-4.jpg', title: 'Paris Runway Pulse' },
  { src: '/assets/fashion-reel/look-5.jpg', title: 'Structured Silhouette Study' },
  { src: '/assets/fashion-reel/look-6.jpg', title: 'High Contrast Utility' },
  { src: '/assets/fashion-reel/look-7.jpg', title: 'Modern Couture Motion' },
  { src: '/assets/fashion-reel/look-8.jpg', title: 'Heritage x Future Blend' }
]

const LOOP_IMAGES = [...REEL_IMAGES, ...REEL_IMAGES]

export default function CameraReelPlaceholder() {
  return (
    <section className="camera-reel-placeholder" aria-label="Moving fashion camera reel showcase">
      <div className="camera-reel-head">
        <p className="camera-reel-label">Live Camera Reel</p>
        <p className="camera-reel-meta">Continuous fashion feed // right to left loop</p>
      </div>

      <div className="camera-reel-window">
        <div className="camera-reel-track">
          {LOOP_IMAGES.map((image, index) => (
            <figure className="camera-frame" key={`${image.src}-${index}`}>
              <img className="camera-frame-image" src={image.src} alt={image.title} loading="lazy" />
              <figcaption className="camera-frame-caption">{image.title}</figcaption>
            </figure>
          ))}
        </div>
      </div>

      <style jsx>{`
        .camera-reel-placeholder {
          position: relative;
          overflow: hidden;
          border: 1px solid rgba(255, 255, 255, 0.22);
          background:
            linear-gradient(180deg, rgba(255, 255, 255, 0.06), rgba(255, 255, 255, 0.01)),
            linear-gradient(110deg, rgba(0, 229, 255, 0.08), rgba(0, 0, 0, 0.2));
          box-shadow: 0 26px 70px rgba(0, 0, 0, 0.56);
        }

        .camera-reel-placeholder::before,
        .camera-reel-placeholder::after {
          content: '';
          position: absolute;
          left: 0;
          width: 100%;
          height: 20px;
          z-index: 2;
          background-image: radial-gradient(circle, rgba(0, 0, 0, 0.9) 39%, transparent 41%);
          background-size: 18px 18px;
          background-position: center;
          background-color: rgba(4, 4, 6, 0.9);
        }

        .camera-reel-placeholder::before {
          top: 0;
        }

        .camera-reel-placeholder::after {
          bottom: 0;
        }

        .camera-reel-head {
          position: relative;
```

```
  z-index: 3;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1rem;
  padding: 2rem 1.2rem 0.9rem;
  border-bottom: 1px solid rgba(255, 255, 255, 0.12);
}

.camera-reel-label {
  font-family: var(--font-heading);
  text-transform: uppercase;
  letter-spacing: 0.22em;
  font-size: 0.7rem;
  color: rgba(255, 255, 255, 0.95);
}

.camera-reel-meta {
  font-family: var(--font-mono);
  text-transform: uppercase;
  letter-spacing: 0.17em;
  font-size: 0.58rem;
  color: rgba(255, 255, 255, 0.62);
}

.camera-reel-window {
  position: relative;
  overflow: hidden;
  padding: 1rem 0 2.2rem;
}

.camera-reel-track {
  display: flex;
  align-items: stretch;
  gap: 1rem;
  width: max-content;
  padding: 0 1rem;
  will-change: transform;
  animation: camera-reel-scroll 34s linear infinite;
}

.camera-frame {
  width: clamp(220px, 24vw, 360px);
  margin: 0;
  border: 1px solid rgba(255, 255, 255, 0.16);
  background: rgba(3, 3, 4, 0.92);
}

.camera-frame-image {
  display: block;
  width: 100%;
  height: clamp(160px, 18vw, 240px);
  object-fit: cover;
  filter: saturate(1.08) contrast(1.03);
}

.camera-frame-caption {
  padding: 0.72rem;
  font-family: var(--font-mono);
  text-transform: uppercase;
  letter-spacing: 0.16em;
  font-size: 0.58rem;
  color: rgba(255, 255, 255, 0.82);
  border-top: 1px solid rgba(255, 255, 255, 0.1);
}

@keyframes camera-reel-scroll {
  0% {
    transform: translate3d(0, 0, 0);
  }
  100% {
    transform: translate3d(-50%, 0, 0);
  }
}
```

```
        @media (max-width: 840px) {
          .camera-reel-head {
            align-items: flex-start;
            flex-direction: column;
            gap: 0.35rem;
            padding: 1.8rem 1rem 0.8rem;
          }

          .camera-reel-window {
            padding-bottom: 2rem;
          }

          .camera-frame {
            width: clamp(200px, 62vw, 290px);
          }

          .camera-frame-image {
            height: clamp(145px, 44vw, 210px);
          }
        }
      `}</style>
    </section>
  )
}
```

frontend/components/ImageUpload.tsx

```
import { useState, useCallback } from 'react'
import { useDropzone } from 'react-dropzone'
import { motion } from 'framer-motion'
import axios from 'axios'

const API_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8000'

interface ImageUploadProps {
  onRecommendations: (data: any) => void
  loading: boolean
  setLoading: (loading: boolean) => void
  occasion: string
  style: string
}

export default function ImageUpload({ onRecommendations, loading, setLoading, occasion, style }: ImageUploadProps) {
  const [image, setImage] = useState<File | null>(null)
  const [preview, setPreview] = useState<string | null>(null)
  const [error, setError] = useState<string | null>(null)

  const onDrop = useCallback((acceptedFiles: File[]) => {
    const file = acceptedFiles[0]
    if (file) {
      setImage(file)
      setPreview(URL.createObjectURL(file))
      setError(null)
    }
  }, [])

  const { getRootProps, getInputProps, isDragActive } = useDropzone({
    onDrop,
    accept: {
      'image/*': ['.jpeg', '.jpg', '.png']
    },
    maxFiles: 1,
    maxSize: 10 * 1024 * 1024 // 10MB
  })

  const handleSubmit = async () => {
    if (!image) {
      setError('Please upload an image')
      return
    }

    setLoading(true)
    setError(null)

    const formData = new FormData()
    formData.append('image', image)
    formData.append('occasion', occasion || 'casual')
    formData.append('style', style || 'minimalist')
    formData.append('include_brands', 'false')

    try {
      const response = await axios.post(`${API_URL}/api/analyze`, formData, {
        headers: {
          'Content-Type': 'multipart/form-data',
        },
        timeout: 300000 // 5 minute timeout
      })

      onRecommendations(response.data)
    } catch (err: any) {
      console.error('Error:', err)
      setError(
        err.response?.data?.detail ||
        err.message ||
        'Failed to analyze image. Please try again.'
      )
    } finally {
      setLoading(false)
    }
```

```
  }

  return (
    <div className="space-y-5 w-full">
      {/* Upload Dropzone */}
      <div
        {...getRootProps()}
        className={`
          upload-zone rounded-xl p-10 cursor-pointer text-center
          transition-all duration-300
          ${isDragActive ? 'border-nexus-brown bg-nexus-beige/60' : ''}
        `}
      >
        <input {...getInputProps()} />

        {preview ? (
          <motion.div
            initial={{ opacity: 0, scale: 0.95 }}
            animate={{ opacity: 1, scale: 1 }}
            className="space-y-4"
          >
            <div className="relative w-full max-w-xs mx-auto aspect-[3/4] rounded-lg overflow-hidden shadow-md">
              <img
                src={preview}
                alt="Preview"
                className="w-full h-full object-cover"
              />
            </div>
            <p className="text-sm text-nexus-gray">
              Click or drag to change image
            </p>
          </motion.div>
        ) : (
          <div className="space-y-4 py-4">
            {/* Upload Icon */}
            <div className="mx-auto w-16 h-16 rounded-full bg-nexus-beige flex items-center justify-center">
              <svg className="w-8 h-8 text-nexus-brown" fill="none" stroke="currentColor" strokeWidth="1.5" viewBox="0
0 24 24">
                <path strokeLinecap="round" strokeLinejoin="round" d="M3 16.5v2.25A2.25 2.25 0 005.25 21h13.5A2.25
2.25 0 0021 18.75V16.5m-13.5-9L12 3m0 0l4.5 4.5M12 3v13.5" />
              </svg>
            </div>
            <div className="space-y-1">
              <p className="text-xl font-serif text-nexus-text">
                {isDragActive ? 'Drop your photo here' : 'Upload your photo'}
              </p>
              <p className="text-nexus-gray text-sm">
                Drag & drop or click to browse
              </p>
              <p className="text-xs text-nexus-gray/60">
                JPG, PNG up to 10MB
              </p>
            </div>
          </div>
        )}
      </div>

      {/* Submit Button */}
      <motion.button
        whileTap={{ scale: 0.98 }}
        onClick={handleSubmit}
        disabled={!image || loading}
        className={`
          w-full py-4 px-8 rounded-sm text-xs font-bold tracking-widest uppercase
          transition-all duration-300
          ${!image || loading
            ? 'bg-nexus-beige text-nexus-gray cursor-not-allowed'
            : 'bg-[#A06448] hover:bg-[#8c563e] text-white shadow-md hover:shadow-lg'
          }
        `}
      >
        {loading ? (
          <span className="flex items-center justify-center gap-3">
            <div className="loading-spinner"></div>
            <span>Analyzing your style...</span>
```

```
          </span>
        ) : (
          <span>GET MY RECOMMENDATIONS</span>
        )}
      </motion.button>

      {/* Error Message */}
      {error && (
        <motion.div
          initial={{ opacity: 0, y: -10 }}
          animate={{ opacity: 1, y: 0 }}
          className="bg-red-50 border border-red-200 rounded-lg p-4"
        >
          <div className="flex items-start gap-3">
            <svg className="w-5 h-5 text-red-500 mt-0.5 flex-shrink-0" fill="none" stroke="currentColor"
strokeWidth="2" viewBox="0 0 24 24">
              <path strokeLinecap="round" strokeLinejoin="round" d="M12 9v3.75m9-.75a9 9 0 11-18 0 9 9 0 0118 0zm-9
3.75h.008v.008H12v-.008z" />
            </svg>
            <div>
              <p className="font-semibold text-red-700 text-sm">Something went wrong</p>
              <p className="text-sm text-red-600 mt-0.5">{error}</p>
            </div>
          </div>
        </motion.div>
      )}
    </div>
  )
}
```

frontend/components/RecommendationResults.tsx

```
import { motion } from 'framer-motion'
import Link from 'next/link'

interface RecommendationResultsProps {
  data: any
  onReset: () => void
}

function ClothingImageCard({
  label,
  item,
  imageUrl,
}: {
  label: string
  item?: string
  imageUrl?: string
}) {
  return (
    <div className="overflow-hidden rounded-lg border border-nexus-beige/40 bg-nexus-cream">
      <div className="aspect-[4/5] bg-white">
        {imageUrl ? (
          <img
            src={imageUrl}
            alt={item || label}
            className="h-full w-full object-cover"
            loading="lazy"
            referrerPolicy="no-referrer"
          />
        ) : (
          <div className="flex h-full items-center justify-center px-4 text-center text-xs uppercase tracking-[0.2em]
text-nexus-gray">
            Image unavailable
          </div>
        )}
      </div>
      <div className="p-3">
        <span className="text-[10px] font-bold text-nexus-brown uppercase tracking-widest">{label}</span>
        <p className="text-nexus-text text-sm font-medium mt-1">{item || 'Not specified'}</p>
      </div>
    </div>
  )
}

// Helper function to format text with proper structure
function formatTextWithStructure(text: string) {
  if (!text) return null;

  const lines = text
    .replace(/OUTFIT \d+:/gi, '\n**OUTFIT:**')
    .replace(/- Name:/gi, '\n- **Name:**')
    .replace(/- Description:/gi, '\n- **Description:**')
    .replace(/- Top:/gi, '\n- **Top:**')
    .replace(/- Bottom:/gi, '\n- **Bottom:**')
    .replace(/- Shoes:/gi, '\n- **Shoes:**')
    .replace(/- Colors:/gi, '\n- **Colors:**')
    .replace(/- Why it works:/gi, '\n- **Why it works:**')
    .replace(/However,/gi, '\n\nHowever,')
    .replace(/Since/gi, '\n\nSince')
    .split('\n')
    .filter(line => line.trim());

  return lines.map((line, idx) => {
    const trimmed = line.trim();
    if (trimmed.startsWith('-') || trimmed.startsWith('•')) {
      return (
        <div key={idx} className="flex items-start gap-2 ml-4 my-1">
          <span className="text-nexus-brown mt-1 flex-shrink-0">•</span>
          <span dangerouslySetInnerHTML={{
            __html: trimmed.substring(1).replace(/\*\*(.*?)\*\*/g, '<strong class="text-nexus-brown font-
semibold">$1</strong>')
          }} />
        </div>
```

```
      );
    }
    if (trimmed.startsWith('**')) {
      return (
        <h5 key={idx} className="font-serif font-bold text-nexus-text mt-4 mb-2"
          dangerouslySetInnerHTML={{
            __html: trimmed.replace(/\*\*(.*?)\*\*/g, '$1')
          }} />
      );
    }
    return <p key={idx} className="my-2">{trimmed}</p>;
  });
}

export default function RecommendationResults({ data, onReset }: RecommendationResultsProps) {
  const {
    recommendations,
    processing_time,
    used_api,
    is_appropriate,
    critique,
    improvement_suggestions,
  } = data || {}

  const processingText =
    typeof processing_time === 'number' ? `${processing_time.toFixed(2)}s` : null
  const apiText = typeof used_api === 'string' ? used_api.toUpperCase() : null

  const hasRecommendations = Array.isArray(recommendations) && recommendations.length > 0

  return (
    <div className="space-y-8">
      {/* Header Card */}
      <motion.div
        initial={{ opacity: 0, y: -20 }}
        animate={{ opacity: 1, y: 0 }}
        className="bg-white rounded-xl p-6 md:p-8 shadow-sm border border-nexus-beige/50"
      >
        <div className="flex flex-col md:flex-row justify-between items-start md:items-center gap-6">
          <div>
            <h2 className="font-serif text-3xl md:text-4xl text-nexus-text mb-2">Style Analysis</h2>
            {(processingText || apiText) && (
              <p className="text-nexus-gray text-sm">
                {processingText ? `Completed in ${processingText}` : 'Results generated'}
                {apiText ? ` • ${apiText} AI` : ''}
              </p>
            )}
          </div>
          <motion.button
            whileTap={{ scale: 0.95 }}
            onClick={onReset}
            className="px-6 py-3 bg-nexus-beige/50 hover:bg-nexus-beige text-nexus-text rounded-sm text-xs font-bold
tracking-widest uppercase transition-all duration-300"
          >
            ← New Analysis
          </motion.button>
          <Link
            href="/vton"
            className="px-6 py-3 bg-black hover:bg-[#ff0000] text-white rounded-sm text-xs font-bold tracking-widest
uppercase transition-all duration-300"
          >
            VTON Generator
          </Link>
        </div>
      </motion.div>

      {/* Verdict Section */}
      {(critique || improvement_suggestions) && (
        <motion.div
          initial={{ opacity: 0, y: 20 }}
          animate={{ opacity: 1, y: 0 }}
          transition={{ duration: 0.5, delay: 0.1 }}
          className="bg-white rounded-xl p-6 md:p-8 shadow-sm border border-nexus-beige/50"
        >
          <div class="flex flex-col md:flex-row items-start gap-6">
```

```
            <div
              className={`w-14 h-14 md:w-16 md:h-16 rounded-full flex items-center justify-center text-2xl flex-
shrink-0 ${
                is_appropriate
                  ? 'bg-green-50 text-green-600 border-2 border-green-200'
                  : 'bg-amber-50 text-amber-600 border-2 border-amber-200'
              }`}
            >
              {is_appropriate ? '✓' : '💡'}
            </div>

            <div className="flex-1 space-y-4">
              <h3 className={`font-serif text-2xl md:text-3xl ${is_appropriate ? 'text-green-700' : 'text-amber-
700'}`}>
                {is_appropriate ? 'Perfect Choice!' : 'Style Suggestion'}
              </h3>

              <div className="text-nexus-text leading-relaxed">
                {formatTextWithStructure(critique || "Here is the analysis of your outfit.")}
              </div>

              {improvement_suggestions && (
                <div className="bg-nexus-cream rounded-lg p-5 border border-nexus-beige">
                  <div className="flex items-start gap-3">
                    <span className="text-xl">💡</span>
                    <div className="flex-1">
                      <h4 className="font-serif font-semibold text-nexus-brown mb-3">Styling Tips</h4>
                      <div className="text-nexus-gray leading-relaxed text-sm">
                        {formatTextWithStructure(improvement_suggestions)}
                      </div>
                    </div>
                  </div>
                </div>
              )}
            </div>
          </div>
        </motion.div>
      )}

      {/* OUTFIT RECOMMENDATIONS */}
      {hasRecommendations && (
        <motion.div
          initial={{ opacity: 0, y: 20 }}
          animate={{ opacity: 1, y: 0 }}
          transition={{ duration: 0.5, delay: 0.2 }}
          className="space-y-6"
        >
          <h3 className="font-serif text-2xl md:text-3xl text-nexus-text">
            {is_appropriate ? 'Styling Enhancements' : 'Recommended Outfits'}
          </h3>

          <div className="space-y-6">
            {recommendations.map((rec: any, idx: number) => {
              return (
                <motion.div
                  key={idx}
                  initial={{ opacity: 0, y: 20 }}
                  animate={{ opacity: 1, y: 0 }}
                  transition={{ duration: 0.5, delay: 0.2 + idx * 0.1 }}
                  className="bg-white rounded-xl overflow-hidden shadow-sm border border-nexus-beige/50 hover:shadow-
md transition-shadow duration-300"
                >
                  <div className="p-6 md:p-8 space-y-5">
                      {/* Header */}
                      <div className="flex justify-between items-start gap-4">
                        <div className="flex items-center gap-3">
                          <div className="w-9 h-9 rounded-full bg-nexus-brown flex items-center justify-center text-
white font-bold text-sm">
                            {idx + 1}
                          </div>
                          <h4 className="font-serif text-xl md:text-2xl text-nexus-text">
                            {rec.outfit_name}
                          </h4>
                        </div>
                        {rec.match_score && (
```

```
                        <span className="px-3 py-1.5 bg-nexus-beige/50 rounded-sm text-xs font-bold tracking-wider
text-nexus-brown">
                          {rec.match_score}% MATCH
                        </span>
                      )}
                    </div>

                    <p className="text-nexus-gray leading-relaxed text-sm">{rec.description}</p>

                    {/* Why it works */}
                    {rec.why_it_works && (
                      <div className="bg-nexus-cream rounded-lg p-4 border border-nexus-beige/50">
                        <p className="text-nexus-gray leading-relaxed text-sm">
                          {rec.why_it_works}
                        </p>
                      </div>
                    )}

                    {/* Outfit Items */}
                    <div className="grid grid-cols-1 sm:grid-cols-2 gap-3">
                      <div className="bg-nexus-cream rounded-lg p-3 border border-nexus-beige/30">
                        <span className="text-[10px] font-bold text-nexus-brown uppercase tracking-
widest">Top</span>
                        <p className="text-nexus-text text-sm font-medium mt-1">{rec.top}</p>
                      </div>

                      <div className="bg-nexus-cream rounded-lg p-3 border border-nexus-beige/30">
                        <span className="text-[10px] font-bold text-nexus-brown uppercase tracking-
widest">Bottom</span>
                        <p className="text-nexus-text text-sm font-medium mt-1">{rec.bottom}</p>
                      </div>
                    </div>

                    {(rec.top_image_url || rec.bottom_image_url || rec.shoes_image_url) && (
                      <div className="space-y-3">
                        <p className="text-[10px] font-bold text-nexus-gray uppercase tracking-widest">
                          Recommended Clothing Photos
                        </p>
                        <div className="grid grid-cols-1 gap-4 md:grid-cols-2 xl:grid-cols-3">
                          <ClothingImageCard label="Upper Body" item={rec.top} imageUrl={rec.top_image_url} />
                          <ClothingImageCard label="Lower Body" item={rec.bottom} imageUrl={rec.bottom_image_url} />
                          {rec.shoes && (
                            <ClothingImageCard label="Shoes" item={rec.shoes} imageUrl={rec.shoes_image_url} />
                          )}
                        </div>
                      </div>
                    )}

                    {/* Color Palette */}
                    {rec.colors && rec.colors.length > 0 && (
                      <div className="pt-4 border-t border-nexus-beige/50">
                        <p className="text-[10px] font-bold text-nexus-gray uppercase tracking-widest mb-2">Color
Palette</p>
                        <div className="flex gap-2 flex-wrap">
                          {rec.colors.map((color: string, colorIdx: number) => (
                            <span
                              key={colorIdx}
                              className="px-3 py-1.5 rounded-sm text-xs font-medium bg-nexus-beige/40 text-nexus-
text border border-nexus-beige"
                            >
                              {color}
                            </span>
                          ))}
                        </div>
                      </div>
                    )}
                  </div>
                </motion.div>
              )
            })}
          </div>
        </motion.div>
      )}

      {!hasRecommendations && (
```

```
        <motion.div
          initial={{ opacity: 0 }}
          animate={{ opacity: 1 }}
          className="bg-white rounded-xl p-12 text-center shadow-sm border border-nexus-beige/50"
        >
          <p className="text-nexus-gray text-lg font-serif">No recommendations available.</p>
        </motion.div>
      )}
    </div>
  )
}
```